UT2-P01. Back-end de la aplicación de reservas

Programación de aplicaciones utilizando frameworks · 2.º DAW

IES Hontanilla · Tarancón

Curso 2026-2027

Datos de la práctica

Módulo	Programación de aplicaciones utilizando frameworks (2.º DAW)
Unidad	Unidad 2. Creación de aplicaciones con un framework de servidor
Resultado de aprendizaje	RA1. Crea aplicaciones BackEnd utilizando framework de servidor
Criterios de evaluación	`RA1.a`, `RA1.b`, `RA1.c`, `RA1.d`, `RA1.e`, `RA1.f`, `RA1.g`
Instrumento	Práctica del RA, **Apto / No Apto**
Modalidad	Individual, en clase y en casa
Entrega	En el plazo indicado en el aula virtual de EducamosCLM
Segunda entrega (si es No Apta)	En el plazo indicado en el aula virtual de EducamosCLM

1. Qué vais a construir

La aplicación «Reservas»

En un instituto hay espacios que se comparten: aulas, laboratorios, el salón de actos y las pistas deportivas. Hoy, para reservarlos, el profesorado apunta su nombre en una hoja de papel en conserjería, y a menudo dos personas reservan el mismo espacio a la misma hora sin saberlo.

«Reservas» es una aplicación web que sustituye esa hoja de papel. La usan dos tipos de personas:

- **El profesorado**, que consulta qué espacios hay, ve qué reservas confirmadas tiene cada uno, pide reservas para un día y una franja horaria, y revisa el estado de sus peticiones.
- **La conserjería** (el personal del centro), que da de alta y de baja los espacios y **confirma** las reservas pendientes desde un panel de administración.

La aplicación tiene una regla que la hoja de papel no podía garantizar: **no se puede reservar un espacio en una franja que se cruce con otra reserva ya confirmada** de ese mismo espacio y ese mismo día.

Durante el curso, cada uno de vosotros desarrollará su propia versión de «Reservas», de forma individual, y la hará crecer unidad a unidad: en esta práctica construís la parte del **servidor**; en la Unidad 3 le añadiréis una aplicación cliente con Vue (`reservas-web`); en las siguientes la desplegaréis con Docker, mejoraréis su interfaz con PrimeVue y le escribiréis pruebas automáticas.

Lo que podrá hacer al terminar la práctica

Cuando terminéis, al arrancar el servidor con `python manage.py runserver`, la aplicación tendrá estas **páginas web**:

Dirección	Quién la usa	Qué se ve
`/`	Cualquiera	La lista de espacios **activos**, cada uno con su tipo («Aula», «Laboratorio»…) y su aforo, y un enlace a su ficha
`/espacios/1/`	Cualquiera	La ficha del espacio 1: tipo, aforo, si está disponible y sus próximas reservas confirmadas (fecha, horario y quién reservó)
`/cuentas/login/`	Profesorado	El formulario para entrar con usuario y contraseña
`/reservas/nueva/`	Profesorado con sesión iniciada	El formulario para pedir una reserva: espacio, fecha, hora de inicio, hora de fin y motivo. Si la franja choca con una reserva confirmada, muestra el error y no la guarda
`/reservas/mias/`	Profesorado con sesión iniciada	Una tabla con mis reservas: espacio, fecha, horario, motivo y estado («Pendiente», «Confirmada», «Cancelada»)

Dirección	Quién la usa	Qué se ve
`/admin/`	Conserjería	El panel de administración: alta y baja de espacios, confirmación de reservas, filtros y buscador

En todas las páginas hay un menú común: «Espacios» y, según se haya iniciado sesión o no, «Entrar» o bien «Nueva reserva», «Mis reservas», el nombre del usuario y un botón «Salir».

Además, la aplicación ofrecerá una **API REST**: direcciones que no devuelven páginas sino **datos en formato JSON**, para que los usen otros programas (en la Unidad 3, vuestra aplicación Vue). Por ejemplo:

Petición	Qué devuelve
`GET /api/espacios/`	La lista de espacios: `[{"id":1,"nombre":"Aula 12","tipo":"aula","aforo":30,"activo":true}, ...]`
`GET /api/espacios/ocupacion/?fecha=2027-05-20`	Para cada espacio activo, cuántas reservas confirmadas tiene ese día
`POST /api/token/`	A cambio de usuario y contraseña, un **token**: una clave que el programa cliente enviará en cada petición para identificarse
`GET /api/reservas/?fecha=2027-05-20`	Las reservas de ese día (solo con token)
`POST /api/reservas/`	Crea una reserva a nombre del usuario del token, aplicando la misma regla de solapes que el formulario
`PATCH` o `DELETE /api/reservas/7/`	Modifica o borra la reserva 7, solo si es del usuario del token

Las piezas de la aplicación

La aplicación es el **back-end** (la parte que se ejecuta en el servidor) de un sistema de reservas. Está hecha con el framework **Django** y tiene estas piezas, que iréis construyendo de una en una:

Pieza	Qué es, en pocas palabras	Dónde está	Capítulo de la unidad
Proyecto y configuración	La carpeta con los ajustes generales: aplicaciones instaladas, idioma, base de datos	`config/settings.py`	2
Rutas y vistas	Qué código atiende cada dirección y qué responde	`urls.py`, `views.py`	3
Plantillas	El HTML de las páginas, con huecos que se rellenan con datos	`templates/`	3

Pieza	Qué es, en pocas palabras	Dónde está	Capítulo de la unidad
Middleware	Código que se ejecuta en **todas** las peticiones; aquí, para apuntar cada petición en un registro	`middleware.py`	4
Modelos y base de datos	La descripción de los datos (espacios y reservas) y la base de datos SQLite donde se guardan	`models.py`, `migrations/`, `db.sqlite3`	5
Consultas	Las preguntas a la base de datos («¿cuántas reservas confirmadas tiene cada espacio el día X?»)	`consultas.py`, vistas	6
Panel de administración	Las pantallas de gestión que Django genera a partir de los modelos	`admin.py`	6
Formularios	La validación de lo que escribe el usuario	`forms.py`	6
API REST	Las direcciones que devuelven JSON, con Django REST Framework	`serializers.py`, `api.py`	7
Seguridad	Quién puede hacer qué: sesiones, tokens, permisos, protección CSRF y CORS	`permissions.py`, ajustes	8

El recorrido de una petición es siempre el mismo: el navegador (o un programa) pide una dirección; la petición pasa por el **middleware**; las **rutas** eligen una **vista**; la vista consulta los **modelos**, que leen o escriben en la **base de datos**; y la vista devuelve una página generada con una **plantilla** o, en la API, un documento JSON.

Cómo está organizado este guion

La práctica se construye en **once fases**. Cada fase añade una pieza y termina con la aplicación funcionando y con un apartado **Comprobación de la fase** que indica el resultado exacto que debéis obtener. No paséis a la fase siguiente sin haber obtenido ese resultado. Cada fase indica qué capítulo del documento de la Unidad 2 hay que haber estudiado antes.

Fase	Qué se consigue	Capítulo previo	CE
1	Proyecto creado y configurado	2	`RA1.a`
2	Primeras rutas y vistas, con datos en memoria	3.1 a 3.4	`RA1.b`
3	Páginas con plantillas	3.5 a 3.7	`RA1.b`
4	Registro de peticiones con un middleware propio	4	`RA1.c`
5	Modelos, regla de solapes y migraciones; las páginas leen la base de datos	5	`RA1.d`

Fase	Qué se consigue	Capítulo previo	CE
6	Panel de administración y datos de prueba	6.1 y 6.3	`RA1.d`, `RA1.e`
7	Páginas definitivas y consultas	6.2 y 6.4	`RA1.b`, `RA1.e`
8	Inicio de sesión, formulario de nueva reserva y «Mis reservas»	6.5 y 8.2	`RA1.b`, `RA1.g`
9	API REST	7	`RA1.e`, `RA1.f`
10	Seguridad de la API: token, permisos y CORS	8	`RA1.g`
11	Repaso final y entrega	Todo	Todos

Las fases 1 a 4 se pueden hacer con lo visto en los capítulos 2 a 4; a partir de la fase 5 cada fase sigue el ritmo de la unidad. Lo que no terminéis en clase se termina en casa.

Cómo trabajar

- **Entorno.** Windows con **Python 3.13** (válidas de la 3.10 a la 3.13), Visual Studio Code y la terminal PowerShell, como en la unidad. Las órdenes se escriben en la carpeta del proyecto con el entorno virtual activado: el indicador debe empezar por `(.venv)`. Cuando un bloque muestra `>` delante de una orden, el `>` no se escribe.
- **Lo que os damos y lo que escribís vosotros.** El guion da completo el código de configuración y el que introduce algo nuevo. La lógica propia de la aplicación (las reglas de las reservas, las consultas, los permisos) se describe con una especificación precisa y unas pistas: la escribís vosotros. Si algo no os sale, volved al apartado de la unidad que se indica: allí está el mismo mecanismo aplicado a «Incidencias».
- **Comprobad cada fase.** Las salidas de la **Comprobación de la fase** son las que se obtienen con una solución correcta. Si la vuestra no coincide, revisad antes los **errores frecuentes** de la fase.
- **Guardad vuestro trabajo.** Si usáis Git, haced al menos un *commit* al terminar cada fase (por ejemplo, `Fase 3: plantillas`).

Fase 1. Proyecto y configuración

Qué vais a conseguir. El esqueleto del proyecto: la carpeta `reservas` con su entorno virtual, el proyecto Django `config`, la aplicación `reservas` y los ajustes de idioma y zona horaria. Instalaréis ya los tres paquetes que usaréis en toda la práctica.

Antes de empezar. Capítulo 2 de la unidad (proyecto y aplicaciones, `settings.py`, `manage.py`, `.gitignore`).

Pasos

1. Cread la carpeta del proyecto y su entorno virtual, e instalad los paquetes:

```
> cd $HOME\proyectos\pauf
> mkdir reservas
> cd reservas
> python -m venv .venv
> .\.venv\Scripts\Activate.ps1
> python -m pip install --upgrade pip
> python -m pip install "django~=5.2.0" "djangorestframework~=3.16.0"
 ↪  "django-cors-headers~=4.9"
```

2. Cread el proyecto (con el punto final) y la aplicación:

```
> django-admin startproject config .
> python manage.py startapp reservas
```

3. En `config/settings.py`, añadid al final de `INSTALLED_APPS` vuestra aplicación y las tres de los paquetes instalados, y cambiad el idioma y la zona horaria:

```
# config/settings.py (fragmentos)
INSTALLED_APPS = [
    # ... las seis de django.contrib ...
    "reservas",
    "rest_framework",
    "rest_framework.authtoken",
    "corsheaders",
]

LANGUAGE_CODE = "es-es"
TIME_ZONE = "Europe/Madrid"
```

4. Cread las tablas de las aplicaciones de serie, comprobad la configuración y guardad las dependencias:

```
> python manage.py migrate
> python manage.py check
> pip freeze > requirements.txt
```

5. Cread el fichero `.gitignore` en la carpeta del proyecto:

```
.venv/
__pycache__/
*.pyc
db.sqlite3
peticiones.log
.vscode/
```

6. Arrancad el servidor con `python manage.py runserver` y abrid http://127.0.0.1:8000/ y http://127.0.0.1:8000/admin/.

Comprobación de la fase

- `migrate` aplica, entre otras, `authtoken.0001_initial` a `authtoken.0004_alter_tokenproxy_options` y termina con `Applying sessions.0001_initial... OK`.
- `check` responde `System check identified no issues (0 silenced).`
- `requirements.txt` contiene, entre otras, las líneas `Django==5.2.x`, `djangorestframework==3.16.x` y `django-cors-headers==4.9.x`.
- `/` muestra la página del cohete con el título de pestaña «¡La instalación funcionó con éxito! ¡Felicitaciones!» (ya en español).
- `/admin/` redirige al formulario de acceso con el título «Iniciar sesión | Sitio de administración de Django».

Errores frecuentes

Síntoma	Causa	Solución
Aparece `config\config\`	Faltó el punto final en `startproject`	Borrar la carpeta y repetir con el punto
`ModuleNotFoundError: No module named 'corsheaders'`	Paquete instalado fuera del entorno virtual	Activar `.venv` y repetir la instalación
La página del cohete aparece en inglés	`LANGUAGE_CODE` sin cambiar o mal escrito	`LANGUAGE_CODE = "es-es"`
`No matching distribution found for django~=5.2.0`	Python anterior a 3.10	Comprobar `python --version` dentro del entorno

Fase 2. Primeras rutas y vistas

Qué vais a conseguir. Las dos primeras direcciones de la aplicación: `/`, que responde con la lista de espacios, y `/espacios/<id>/`, que responde con los datos de un espacio. Como todavía no hay base de datos, los espacios se guardan en una **lista en memoria**; en la fase 5 se sustituirá por consultas.

Antes de empezar. Apartados 3.1 a 3.4 (rutas, convertidores, `include`, `app_name`, vistas de función, `Http404`).

Pasos

1. Escribid las vistas:

```
# reservas/views.py
from django.http import Http404, HttpResponse

ESPACIOS = [
    {"id": 1, "nombre": "Aula 12", "tipo": "aula", "aforo": 30},
    {"id": 2, "nombre": "Laboratorio de Física", "tipo": "laboratorio",
 ↪  "aforo": 24},
```

```
    {"id": 3, "nombre": "Salón de actos", "tipo": "salon", "aforo": 200},
]


def buscar(pk):
    for espacio in ESPACIOS:
        if espacio["id"] == pk:
            return espacio
    raise Http404("El espacio no existe")


def lista(request):
    nombres = ", ".join(e["nombre"] for e in ESPACIOS)
    return HttpResponse(f"{len(ESPACIOS)} espacios: {nombres}")


def detalle(request, pk):
    espacio = buscar(pk)
    return HttpResponse(f"{espacio['nombre']} ({espacio['tipo']}), aforo
    ↪ {espacio['aforo']}")
```

2. Cread `reservas/urls.py`. Los nombres `lista` y `detalle`, dentro del espacio de nombres `reservas`, se usarán en toda la práctica:

```
# reservas/urls.py
from django.urls import path

from . import views

app_name = "reservas"

urlpatterns = [
    path("", views.lista, name="lista"),
    path("espacios/<int:pk>/", views.detalle, name="detalle"),
]
```

3. Incluid las rutas de la aplicación en el proyecto:

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("reservas.urls")),
]
```

4. Arrancad el servidor y abrid `/`, `/espacios/2/`, `/espacios/9/`, `/espacios/abc/` y `/espacios/2` (sin la barra final).

Comprobación de la fase

- `/` muestra `3 espacios: Aula 12, Laboratorio de Física, Salón de actos`.
- `/espacios/2/` muestra `Laboratorio de Física (laboratorio), aforo 24`.
- `/espacios/9/` devuelve `404` con el mensaje «El espacio no existe»: la ruta coincide, pero la vista lanza `Http404`.
- `/espacios/abc/` devuelve `404` con la página «Page not found» y la lista de patrones probados: `abc` no es un entero y la vista no se ejecuta.
- `/espacios/2` devuelve `301 Moved Permanently` con `Location: /espacios/2/` (lo hace `CommonMiddleware`).

Si en la sesión 9 hicisteis el paso opcional «Filtrar por tipo», podéis dejarlo: no forma parte de los requisitos.

Errores frecuentes

Síntoma	Causa
`/` responde con la página del cohete	Falta `include("reservas.urls")` en `config/urls.py`
`NameError: name 'include' is not defined`	Falta importarlo: `from django.urls import include, path`
404 en `/espacios/2/`	La ruta empieza por barra (`"/espacios/..."`) o falta la barra final
`TypeError: detalle() got an unexpected keyword argument 'pk'`	El parámetro de la vista no se llama igual que en la ruta

Fase 3. Páginas con plantillas

Qué vais a conseguir. Que las dos direcciones devuelvan **páginas HTML** generadas con plantillas que comparten una plantilla base con el menú. Los datos siguen en memoria.

Antes de empezar. Apartados 3.5 a 3.7 (plantillas, `render`, herencia, `{% url %}`, filtros).

Pasos

1. Cambiad las vistas para que usen `render()` y una plantilla. Sustituid en `reservas/views.py` la importación de `HttpResponse` y las dos vistas:

```
# reservas/views.py (cambios)
from django.http import Http404
from django.shortcuts import render

# ... ESPACIOS y buscar() no cambian ...
```

```
def lista(request):
    return render(request, "reservas/lista.html", {"espacios": ESPACIOS})


def detalle(request, pk):
    return render(request, "reservas/detalle.html", {"espacio": buscar(pk)})
```

2. Cread la plantilla base. Fijaos en la ruta: `reservas\templates\reservas\base.html`.

```
{# reservas/templates/reservas/base.html #}
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>{% block titulo %}Reservas{% endblock %} | IES Hontanilla</title>
</head>
<body>
  <header>
    <a href="{% url 'reservas:lista' %}">Espacios</a>
  </header>
  <main>
    {% block contenido %}{% endblock %}
  </main>
</body>
</html>
```

3. Cread la plantilla del listado:

```
{# reservas/templates/reservas/lista.html #}
{% extends "reservas/base.html" %}

{% block titulo %}Espacios{% endblock %}

{% block contenido %}
<h1>Espacios disponibles</h1>
<ul>
  {% for espacio in espacios %}
    <li>
      <a href="{% url 'reservas:detalle' espacio.id %}">{{ espacio.nombre
      ↪ }}</a>
      ({{ espacio.tipo }}, aforo {{ espacio.aforo }})
    </li>
  {% empty %}
    <li>No hay espacios disponibles.</li>
  {% endfor %}
</ul>
{% endblock %}
```

4. Escribid vosotros `reservas/templates/reservas/detalle.html`. **Especificación:** hereda de la base; el título de la pestaña es el nombre del espacio; muestra el nombre en un `<h1>`, un párrafo con el texto `Tipo: <tipo> · Aforo: <aforo>` y un enlace «Volver» al listado generado con `{% url %}`.

5. **Reiniciad el servidor** (Django no detecta la carpeta `templates` nueva con el servidor en marcha) y abrid `/` y `/espacios/3/`.

Comprobación de la fase

- La pestaña de `/` muestra «Espacios | IES Hontanilla»; la página tiene el menú «Espacios», el encabezado «Espacios disponibles» y tres enlaces. En el código fuente (`Ctrl + U`) aparecen ya resueltos: `<a href="/espacios/1/">Aula 12</a>` seguido de `(aula, aforo 30)`, y lo mismo para los otros dos espacios.
- `/espacios/3/` tiene la pestaña «Salón de actos | IES Hontanilla», el encabezado «Salón de actos», el párrafo `Tipo: salon · Aforo: 200` y el enlace «Volver», que lleva a `/`.
- `/espacios/9/` sigue devolviendo 404.

Errores frecuentes

Síntoma	Causa
`TemplateDoesNotExist: reservas/lista.html`	La carpeta no es `reservas\templates\reservas\`, la aplicación no está en `INSTALLED_APPS` o no se reinició el servidor
`NoReverseMatch`	Nombre de ruta mal escrito, o falta el espacio de nombres (`'reservas:detalle'`) o el parámetro
La página sale sin menú	Falta `{% extends %}` o no es la primera etiqueta
El texto aparece fuera de su sitio	Contenido escrito fuera de `{% block contenido %}`

Fase 4. Registro de peticiones

Qué vais a conseguir. Un **middleware** propio que apunte en el fichero `peticiones.log` una línea por cada petición que recibe la aplicación, con la fecha y hora, el método, la ruta, el código de estado y el usuario. Es un registro que, en un servidor real, permite saber qué ha pasado y quién lo ha hecho.

Antes de empezar. Capítulo 4 (capas, orden de `MIDDLEWARE`, middleware como clase, `logging`, `request.user`).

Pasos

1. Cread `reservas/middleware.py` con la clase `RegistroPeticionesMiddleware`. **Especificación:**

   - Sigue la estructura de clase del apartado 4.3: `__init__(self, get_response)` y `__call__(self, request)`.

- Llama a `get_response` para obtener la respuesta y, **después**, escribe en el registro una línea con el método, la ruta, el código de estado de la respuesta y el nombre del usuario, o `anonimo` si no ha iniciado sesión. Separad los cuatro datos con un espacio.
- Usa un registrador de `logging` llamado `reservas.peticiones`: `logger = logging.getLogger("reservas.peticiones")` al principio del fichero y `logger.info(...)` para escribir. No uséis `print()`.
- Pistas: `request.user.is_authenticated` indica si hay sesión y `request.user.username` da el nombre; `response.status_code` da el código.

2. Activadlo añadiendo `"reservas.middleware.RegistroPeticionesMiddleware"` al **final** de `MIDDLEWARE`, después de `AuthenticationMiddleware` (apartado 4.2: el usuario lo añade esa capa).

3. Configurad el registro para que escriba en el fichero `peticiones.log` de la carpeta del proyecto. Añadid al final de `settings.py`:

```
# config/settings.py (añadido al final)
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "simple": {"format": "{asctime} {message}", "style": "{"},
    },
    "handlers": {
        "fichero": {
            "class": "logging.FileHandler",
            "filename": BASE_DIR / "peticiones.log",
            "encoding": "utf-8",
            "formatter": "simple",
        },
    },
    "loggers": {
        "reservas.peticiones": {"handlers": ["fichero"], "level": "INFO",
        ↪  "propagate": False},
    },
}
```

4. Reiniciad el servidor, abrid `/`, `/espacios/9/` y `/admin/`, y mirad el final del fichero:

```
> Get-Content peticiones.log -Tail 3
```

Comprobación de la fase

Las tres últimas líneas del fichero tienen este formato (la fecha y la hora serán las vuestras):

```
2026-10-27 23:43:50,416 GET / 200 anonimo
2026-10-27 23:43:50,464 GET /espacios/9/ 404 anonimo
2026-10-27 23:43:50,513 GET /admin/ 302 anonimo
```

En la fase 8, cuando inicies sesión, aparecerá el nombre del usuario en lugar de `anonimo`.

Errores frecuentes

Síntoma	Causa
El servidor no arranca: `ImportError` o `ModuleNotFoundError`	Ruta de la clase mal escrita en `MIDDLEWARE`
`AttributeError: 'WSGIRequest' object has no attribute 'user'`	Se lee `request.user` antes de `get_response` y el middleware está antes de `AuthenticationMiddleware`
`peticiones.log` no se crea o está vacío	Falta `LOGGING`, o el nombre del registrador no coincide (`getLogger(__name__)` da `reservas.middleware`, no `reservas.peticiones`)
Las líneas salen en la terminal y no en el fichero	Se ha usado `print()`

Fase 5. Modelos, regla de solapes y migraciones

Qué vais a conseguir. La base de datos de la aplicación: los modelos `Espacio` y `Reserva`, la regla de negocio de las reservas y sus migraciones. Las páginas dejan de usar la lista en memoria y pasan a leer la base de datos.

Antes de empezar. Capítulo 5 (campos, opciones, `TextChoices`, `ForeignKey` y `on_delete`, `Meta`, migraciones) y apartado 6.5, «Validación en el modelo» (método `clean()`).

Pasos

1. Escribid en `reservas/models.py` los dos modelos con estas especificaciones.

   `Espacio`

Campo	Tipo y opciones
`nombre`	Texto de hasta 80 caracteres, único
`tipo`	Opciones cerradas (`TextChoices` llamada `Tipo`): `aula` («Aula»), `laboratorio` («Laboratorio»), `salon` («Salón de actos»), `pista` («Pista deportiva»)
`aforo`	Entero positivo, como mínimo 1 (`PositiveIntegerField` con `validators=[MinValueValidator(1)]`, de `django.core.validators`)
`activo`	Booleano, por defecto verdadero

   Orden por defecto: por nombre. `__str__` devuelve el nombre. `get_absolute_url()` devuelve la dirección de su ficha con `reverse("reservas:detalle", args=[self.pk])`.

   `Reserva`

Campo	Tipo y opciones
`espacio`	Clave ajena a `Espacio`; **no** se puede borrar un espacio que tenga reservas; `related_name="reservas"`
`usuario`	Clave ajena a `settings.AUTH_USER_MODEL`; al borrar el usuario se borran sus reservas; `related_name="reservas"`
`fecha`	Fecha
`hora_inicio`, `hora_fin`	Horas (`TimeField`)
`motivo`	Texto de hasta 200 caracteres
`estado`	Opciones cerradas (`TextChoices` llamada `Estado`): `pendiente` («Pendiente»), `confirmada` («Confirmada»), `cancelada` («Cancelada»); por defecto `pendiente`
`creada`	Fecha y hora de creación, automática

Orden por defecto: por fecha y hora de inicio. `__str__` devuelve, por ejemplo, `Aula 12 20/05/2027 09:00-10:00` (pista: en una f-string, `{self.fecha:%d/%m/%Y}` y `{self.hora_inicio:%H:%M}`).

2. Añadid a `Reserva` las **reglas de negocio** en su método `clean()`. Una reserva solo es válida si se cumplen las tres condiciones, con estos mensajes exactos:

Regla	Campo del error	Mensaje
La hora de fin es posterior a la de inicio	`hora_fin`	La hora de fin debe ser posterior a la de inicio.
El espacio está activo	`espacio`	El espacio no está disponible para reservas.
Si la reserva no está cancelada, no se solapa con **otra** reserva **confirmada** del mismo espacio y el mismo día	Error general (sin campo)	El espacio ya tiene una reserva confirmada que se solapa con ese horario.

Pistas:

- Un error asociado a un campo se lanza con `raise ValidationError({"hora_fin": "..."})`; un error general, con `raise ValidationError("...")`. `ValidationError` se importa de `django.core.exceptions`.
- Dos franjas se solapan si **cada una empieza antes de que termine la otra**: otra reserva se solapa con esta si `otra.hora_inicio < self.hora_fin` y `otra.hora_fin > self.hora_inicio`. Con desigualdades estrictas, las reservas contiguas (una acaba a las 10:00 y otra empieza a las 10:00) no se solapan. En una consulta: `hora_inicio__lt=self.hora_fin, hora_fin__gt=self.hora_inicio`.

- Filtrad además por el mismo espacio (`espacio_id=self.espacio_id`), la misma fecha y `estado=Reserva.Estado.CONFIRMADA`, y excluid la propia reserva con `.exclude(pk=self.pk)`, para que al modificar una reserva confirmada no choque consigo misma. Conviene escribir esa consulta en un método aparte, `solapes(self)`, y usar `.exists()` en `clean()`.
- `clean()` también se ejecuta cuando faltan datos del formulario. Antes de usar un campo, comprobad que tiene valor (`if self.hora_inicio and self.hora_fin and ...`), y usad `self.espacio_id` para saber si hay espacio antes de acceder a `self.espacio`.

3. (Calidad técnica, recomendado.) Añadid en `Meta` una restricción de la base de datos que garantice que `hora_fin` es posterior a `hora_inicio`:

```
# reservas/models.py (dentro de class Meta de Reserva)
constraints = [
    models.CheckConstraint(
        condition=models.Q(hora_fin__gt=models.F("hora_inicio")),
        name="reserva_fin_posterior_inicio",
    ),
]
```

4. Generad y aplicad las migraciones, y comprobad el SQL:

```
> python manage.py makemigrations
> python manage.py migrate
> python manage.py showmigrations reservas
> python manage.py sqlmigrate reservas 0001
```

5. Sustituid las vistas para que lean la base de datos (la lista `ESPACIOS` y la función `buscar` desaparecen):

```
# reservas/views.py
from django.shortcuts import get_object_or_404, render

from .models import Espacio


def lista(request):
    espacios = Espacio.objects.filter(activo=True)
    return render(request, "reservas/lista.html", {"espacios": espacios})


def detalle(request, pk):
    espacio = get_object_or_404(Espacio, pk=pk)
    return render(request, "reservas/detalle.html", {"espacio": espacio})
```

   Las plantillas no cambian: `espacio.nombre` funciona igual con un diccionario que con un objeto.

6. Probad las reglas desde la consola (`python manage.py shell`). `full_clean()` ejecuta todas las validaciones del modelo, incluida `clean()`:

```
>>> from datetime import date, time
>>> e = Espacio.objects.create(nombre="Aula 12", tipo="aula", aforo=30)
>>> Espacio.objects.create(nombre="Aula 3", tipo="aula", aforo=25,
 ↪  activo=False)
>>> u = User.objects.create_user("profe1", password="Clave.2026")
>>> r1 = Reserva.objects.create(espacio=e, usuario=u, fecha=date(2027, 5, 20),
 ↪  hora_inicio=time(9, 0), hora_fin=time(10, 0), motivo="Examen",
 ↪  estado="confirmada")
>>> print(r1)
>>> Reserva(espacio=e, usuario=u, motivo="Prueba", fecha=date(2027, 5, 20),
 ↪  hora_inicio=time(9, 30), hora_fin=time(10, 30)).full_clean()
>>> Reserva(espacio=e, usuario=u, motivo="Prueba", fecha=date(2027, 5, 20),
 ↪  hora_inicio=time(10, 0), hora_fin=time(11, 0)).full_clean()
>>> Reserva(espacio=e, usuario=u, motivo="Prueba", fecha=date(2027, 5, 20),
 ↪  hora_inicio=time(11, 0), hora_fin=time(10, 0)).full_clean()
>>> Reserva(espacio=Espacio.objects.get(nombre="Aula 3"), usuario=u,
 ↪  motivo="Prueba", fecha=date(2027, 5, 20), hora_inicio=time(9, 0),
 ↪  hora_fin=time(10, 0)).full_clean()
>>> r1.full_clean()
```

Comprobación de la fase

- `makemigrations` muestra `reservas\migrations\0001_initial.py` con `+ Create model Espacio` y `+ Create model Reserva`; `showmigrations reservas` muestra `[X] 0001_initial`.
- `sqlmigrate reservas 0001` contiene `"nombre" varchar(80) NOT NULL UNIQUE`, las claves ajenas `"espacio_id" bigint NOT NULL REFERENCES "reservas_espacio" ("id")` y `"usuario_id" integer NOT NULL REFERENCES "auth_user" ("id")` y, si hicisteis el paso 3, `CONSTRAINT "reserva_fin_posterior_inicio" CHECK ("hora_fin" > ("hora_inicio"))`.
- Antes de crear datos, `/` muestra «No hay espacios disponibles.».
- En la consola: `print(r1)` muestra `Aula 12 20/05/2027 09:00-10:00`. El primer `full_clean()` (09:30-10:30) lanza `ValidationError: {'__all__': ['El espacio ya tiene una reserva confirmada que se solapa con ese horario.']}`; el segundo (10:00-11:00, contigua) no lanza nada; el tercero lanza `{'hora_fin': ['La hora de fin debe ser posterior a la de inicio.']}`; el cuarto, `{'espacio': ['El espacio no está disponible para reservas.']}`; y `r1.full_clean()` no lanza nada (no choca consigo misma).
- Con esos datos, `/` muestra solo «Aula 12» (el «Aula 3» está inactivo) y su ficha funciona.

Errores frecuentes

Síntoma	Causa
`makemigrations` responde `No changes detected`	La aplicación no está en `INSTALLED_APPS` o `models.py` no se ha guardado

Síntoma	Causa
La reserva contigua (10:00-11:00) da error de solape	Se ha usado `__lte` o `__gte` en lugar de `__lt` y `__gt`
`r1.full_clean()` da error de solape	Falta `.exclude(pk=self.pk)`
`RelatedObjectDoesNotExist` en `clean()`	Se usa `self.espacio` sin comprobar antes `self.espacio_id`
`TypeError: '<' not supported between instances of 'NoneType'...`	`clean()` compara horas sin comprobar que tienen valor
Se ha editado `0001_initial.py` a mano	Nunca: se borra el fichero y se vuelve a generar mientras no se haya compartido

Fase 6. Panel de administración y datos de prueba

Qué vais a conseguir. Las pantallas de gestión de la conserjería (el panel de administración) y una orden `cargar_datos` que deja la base de datos siempre con los mismos datos de prueba. Con ellos se hacen todas las comprobaciones siguientes y la corrección.

Antes de empezar. Apartados 6.1 (consola y órdenes propias) y 6.3 (panel de administración).

Pasos

1. Registrad los dos modelos en `reservas/admin.py` con `@admin.register` y una clase `ModelAdmin` para cada uno. **Especificación:**

   - Espacios: columnas nombre, tipo, aforo y activo; filtros por tipo y activo; buscador por nombre.
   - Reservas: columnas espacio, fecha, hora de inicio, hora de fin, usuario y estado; filtros por estado, espacio y fecha; buscador por motivo y por nombre de usuario (`"usuario__username"`); `list_select_related` con el espacio y el usuario; `creada` de solo lectura.

2. Cread la orden `cargar_datos` (estructura del apartado 6.1: carpetas `management/commands` con sus dos `__init__.py` vacíos y el fichero `cargar_datos.py` con una clase `Command`). **Especificación:** borra todas las reservas y deja la base de datos exactamente con estos datos; si los usuarios o los espacios ya existen, no los duplica (pistas: `get_or_create` con `defaults` para los usuarios y `update_or_create` para los espacios; `set_password` para la contraseña). Los espacios se crean en el orden de la tabla. Termina mostrando `Cargados 5 espacios y 6 reservas`.

Usuario	Contraseña	Personal (`is_staff`)
`profe1`	`Clave.2026`	No
`profe2`	`Clave.2026`	No
`conserje`	`Clave.2026`	Sí

Espacio	Tipo	Aforo	Activo
Aula 12	aula	30	Sí
Laboratorio de Física	laboratorio	24	Sí
Salón de actos	salon	200	Sí
Pista polideportiva	pista	60	Sí
Aula 3	aula	25	No

Espacio	Fecha	Horario	Usuario	Estado	Motivo
Aula 12	20/05/2027	09:00-10:00	profe1	confirmada	Examen de Matemáticas
Aula 12	20/05/2027	10:00-11:00	profe2	confirmada	Tutoría de 2.º DAW
Salón de actos	20/05/2027	12:00-14:00	profe1	pendiente	Charla de orientación
Laboratorio de Física	24/05/2027	08:30-10:20	profe2	confirmada	Práctica de óptica
Pista polideportiva	24/05/2027	11:15-12:10	profe1	cancelada	Torneo de recreo
Aula 12	24/05/2027	09:00-10:00	profe1	pendiente	Recuperación

Las fechas están en el futuro para que las reservas aparezcan siempre como «próximas».

3. Partid de una **base de datos nueva**, para que los espacios tengan los identificadores 1 a 5 que usan las comprobaciones: con el servidor parado, borrad `db.sqlite3` y ejecutad:

```
> python manage.py migrate
> python manage.py cargar_datos
> python manage.py cargar_datos
> python manage.py createsuperuser
```

Cread el superusuario `admin` con una contraseña que recordéis.

4. Entrad en http://127.0.0.1:8000/admin/ con el superusuario. En «Reservas», probad los filtros y el buscador. Después, añadid desde el panel una reserva **confirmada** del Aula 12 el 20/05/2027 de 09:30 a 10:30 para `profe2`.

Comprobación de la fase

- Las dos ejecuciones de `cargar_datos` muestran `Cargados 5 espacios y 6 reservas`; en el panel hay 4 usuarios (los 3 de la tabla y `admin`) y 5 espacios.
- `python manage.py help` muestra la sección `[reservas]` con la orden `cargar_datos`.
- `/` muestra, en este orden, `Aula 12` (`/espacios/1/`), `Laboratorio de Física` (`/espacios/2/`), `Pista polideportiva` (`/espacios/4/`) y `Salón de actos` (`/espacios/3/`).

- El listado de reservas del panel muestra 6 reservas, con los filtros «Por estado», «Por espacio» y «Por fecha» y la caja de búsqueda.
- La reserva del paso 4 no se guarda: el panel muestra «El espacio ya tiene una reserva confirmada que se solapa con ese horario.» El panel usa el método `clean()` del modelo.

Errores frecuentes

Síntoma	Causa
`Unknown command: 'cargar_datos'`	Falta algún `__init__.py`, la carpeta no se llama `management\commands` o la aplicación no está instalada
`IntegrityError: UNIQUE constraint failed` en la segunda ejecución	Se crean los usuarios o los espacios con `create()` en lugar de `get_or_create()` o `update_or_create()`
`ProtectedError` al ejecutar la orden	Se borran los espacios antes que las reservas
Los identificadores de los espacios no son 1 a 5	La base de datos no era nueva: los identificadores borrados no se reutilizan
Las reservas no aparecen en el panel	El modelo no está registrado en `admin.py`

Fase 7. Páginas definitivas y consultas

Qué vais a conseguir. Las dos páginas públicas en su versión final, con vistas genéricas, las etiquetas de los tipos («Aula», «Pista deportiva») y las próximas reservas confirmadas en la ficha de cada espacio. Además, la primera consulta con agregación: la **ocupación** de los espacios en una fecha, que la API publicará en la fase 9.

Antes de empezar. Apartados 6.2 (consultas, `Q`, agregaciones, `select_related`) y 6.4 (vistas genéricas).

Pasos

1. Sustituid las vistas de función por vistas genéricas:

```
# reservas/views.py
from django.utils import timezone
from django.views.generic import DetailView, ListView

from .models import Espacio, Reserva


class EspacioListView(ListView):
    template_name = "reservas/lista.html"
```

```
    context_object_name = "espacios"

    def get_queryset(self):
        return Espacio.objects.filter(activo=True)


class EspacioDetailView(DetailView):
    model = Espacio
    template_name = "reservas/detalle.html"
    context_object_name = "espacio"

    def get_context_data(self, **kwargs):
        contexto = super().get_context_data(**kwargs)
        # Completad: la variable "reservas" del contexto (ver la
        ↪ especificación)
        return contexto
```

Especificación de `reservas`: las reservas del espacio (`self.object.reservas`) en estado confirmada y con fecha de hoy en adelante (`fecha__gte=timezone.localdate()`), trayendo el usuario en la misma consulta (`select_related`).

2. Cambiad las rutas para usar las clases (`views.EspacioListView.as_view()` con el nombre `lista` y `views.EspacioDetailView.as_view()` con el nombre `detalle`).

3. Actualizad las plantillas:

   - `lista.html`: el enlace pasa a ser `{{ espacio.get_absolute_url }}` y el tipo se muestra con su etiqueta, `{{ espacio.get_tipo_display }}`.
   - `detalle.html`: el párrafo pasa a ser `<etiqueta del tipo> · Aforo: <aforo> · <Disponible o No disponible>` (pista: filtro `yesno:"Disponible,No disponible"`), y debajo, un `<h2>` «Próximas reservas confirmadas» y una lista con una línea por reserva con el formato `20/05/2027 de 09:00 a 10:00 (profe1)` (filtros `date:"d/m/Y"` y `time:"H:i"`), o «No hay reservas confirmadas.» si no hay ninguna.

4. Cread `reservas/consultas.py` con la función `ocupacion(fecha)`. **Especificación:** devuelve un QuerySet con los espacios **activos**, cada uno con el atributo `reservas_confirmadas` (número de reservas confirmadas de esa fecha, 0 si no tiene ninguna), ordenados de más a menos reservas y, a igualdad, por nombre. Debe ser **una sola consulta**. Pistas: `annotate(reservas_confirmadas=Count("reservas", filter=Q(...)))` (apartado 6.2, «Agregaciones y anotaciones»); si filtráis con `.filter(reservas__fecha=...)` antes de contar, desaparecen los espacios con 0.

5. Probad la consulta y contad las consultas a la base de datos en la consola:

```
>>> from datetime import date
>>> from django.db import connection, reset_queries
>>> from reservas.consultas import ocupacion
>>> reset_queries()
>>> [(e.nombre, e.reservas_confirmadas) for e in ocupacion(date(2027, 5, 20))]
>>> len(connection.queries)
>>> [(e.nombre, e.reservas_confirmadas) for e in ocupacion(date(2027, 5, 24))]
```

Comprobación de la fase

- `/` muestra `Aula 12 (Aula, aforo 30)`, `Laboratorio de Física (Laboratorio, aforo 24)`, `Pista polideportiva (Pista deportiva, aforo 60)` y `Salón de actos (Salón de actos, aforo 200)`.
- `/espacios/1/` muestra `Aula · Aforo: 30 · Disponible` y las líneas `20/05/2027 de 09:00 a 10:00 (profe1)` y `20/05/2027 de 10:00 a 11:00 (profe2)`. `/espacios/5/` muestra `Aula · Aforo: 25 · No disponible` y «No hay reservas confirmadas.». `/espacios/999/` devuelve 404.
- En la consola, la primera lista es `[('Aula 12', 2), ('Laboratorio de Física', 0), ('Pista polideportiva', 0), ('Salón de actos', 0)]` y `len(connection.queries)` vale 1. La segunda es `[('Laboratorio de Física', 1), ('Aula 12', 0), ('Pista polideportiva', 0), ('Salón de actos', 0)]`: la reserva pendiente del Salón y la cancelada de la pista no cuentan.

Errores frecuentes

Síntoma	Causa
El Aula 12 aparece con 3 el día 20/05/2027	`Count("reservas")` sin `filter`, o sin filtrar por estado
Solo aparece el Aula 12 en la ocupación	Se filtra por la fecha antes de `annotate`
La ficha no muestra reservas	Falta el contexto `reservas`, el estado no es `confirmada` o la fecha es anterior a hoy
`AttributeError: 'dict' object has no attribute 'get_tipo_display'`	Quedan vistas que usan la lista en memoria

Fase 8. Inicio de sesión, nueva reserva y «Mis reservas»

Qué vais a conseguir. La parte de la aplicación para el profesorado: entrar con usuario y contraseña, pedir una reserva con un formulario que aplica las reglas de la fase 5, y consultar sus propias reservas.

Antes de empezar. Apartados 6.5 (formularios, `ModelForm`, `CreateView`, `LoginRequiredMixin`, `{% csrf_token %}`) y 8.2 (vistas de autenticación de Django, cierre de sesión por POST).

Pasos

1. Configurad el inicio y el cierre de sesión con las vistas de Django:

   - En `config/urls.py`, añadid `path("cuentas/", include("django.contrib.auth.urls")),` después de la ruta del panel.
   - Al final de `settings.py`:

```
# config/settings.py (añadido al final)
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "reservas:lista"
LOGOUT_REDIRECT_URL = "reservas:lista"
```

- Cread la plantilla del formulario de acceso (sin la subcarpeta `reservas`):

```
{# reservas/templates/registration/login.html #}
{% extends "reservas/base.html" %}

{% block titulo %}Entrar{% endblock %}

{% block contenido %}
<h1>Entrar</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="hidden" name="next" value="{{ next }}">
  <button type="submit">Entrar</button>
</form>
{% endblock %}
```

2. Completad el menú de `base.html`. El cierre de sesión **solo admite POST**, por eso es un formulario:

```
{# reservas/templates/reservas/base.html (cabecera) #}
  <header>
    <a href="{% url 'reservas:lista' %}">Espacios</a>
    {% if user.is_authenticated %}
      | <a href="{% url 'reservas:nueva' %}">Nueva reserva</a>
      | <a href="{% url 'reservas:mias' %}">Mis reservas</a>
      | {{ user.username }}
      <form action="{% url 'logout' %}" method="post" style="display: inline">
        {% csrf_token %}
        <button type="submit">Salir</button>
      </form>
    {% else %}
      | <a href="{% url 'login' %}">Entrar</a>
    {% endif %}
  </header>
```

3. Cread el formulario. Los atributos `type` hacen que el navegador muestre un calendario y un selector de hora; el `format` ISO es necesario para que, si el formulario se vuelve a mostrar con errores, el navegador conserve los valores:

```
# reservas/forms.py
from django import forms

from .models import Espacio, Reserva


class ReservaForm(forms.ModelForm):
```

```
    class Meta:
        model = Reserva
        fields = ["espacio", "fecha", "hora_inicio", "hora_fin", "motivo"]
        labels = {"hora_inicio": "Hora de inicio", "hora_fin": "Hora de fin"}
        widgets = {
            "fecha": forms.DateInput(attrs={"type": "date"},
             ↪  format="%Y-%m-%d"),
            "hora_inicio": forms.TimeInput(attrs={"type": "time"},
             ↪  format="%H:%M"),
            "hora_fin": forms.TimeInput(attrs={"type": "time"},
             ↪  format="%H:%M"),
        }

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["espacio"].queryset = Espacio.objects.filter(activo=True)
```

El `__init__` limita la lista desplegable a los espacios activos. `usuario` y `estado` **no** están en `fields`: no los decide el usuario.

4. Escribid las dos vistas nuevas en `reservas/views.py`. **Especificación:**

   - `ReservaCreateView`: vista de creación protegida (`LoginRequiredMixin`, a la izquierda de `CreateView`) con el formulario `ReservaForm`, la plantilla `reservas/reserva_form.html` y, al guardar, redirección a «Mis reservas» (`success_url = reverse_lazy("reservas:mias")`). El usuario de la reserva es el que ha iniciado sesión (pista: `form_valid`, como `IncidenciaCreateView` en el apartado 6.5). El estado queda en `pendiente` por el valor por defecto.
   - `MisReservasView`: listado protegido con la plantilla `reservas/mis_reservas.html` y la variable `reservas`, que contiene **solo** las reservas del usuario conectado, con su espacio en la misma consulta. El orden es el del modelo.

5. Añadid las rutas `reservas/nueva/` (nombre `nueva`) y `reservas/mias/` (nombre `mias`).

6. Cread las dos plantillas, que heredan de la base:

   - `reserva_form.html`: encabezado «Nueva reserva» y un formulario POST con `{% csrf_token %}`, `{{ form.as_p }}` y un botón «Reservar».
   - `mis_reservas.html`: encabezado «Mis reservas» y una tabla con las columnas Espacio, Fecha, Horario, Motivo y Estado (con su etiqueta: `get_estado_display`), o una fila «No tenéis reservas.» si no hay ninguna.

Comprobación de la fase

Ejecutad `python manage.py cargar_datos` antes de empezar.

- Sin sesión, el menú muestra «Espacios | Entrar». `/reservas/nueva/` y `/reservas/mias/` redirigen a `/cuentas/login/?next=/reservas/nueva/` y `/cuentas/login/?next=/reservas/mias/`.
- Con una contraseña incorrecta, el formulario de acceso muestra «Por favor, introduzca un nombre de usuario y clave correctos. Observe que ambos campos pueden ser sensibles a mayúsculas.»

- Al entrar como `profe1` desde «Mis reservas», se vuelve a `/reservas/mias/` (gracias a `next`) y el menú muestra «Nueva reserva», «Mis reservas», `profe1` y «Salir». La tabla tiene 4 filas, en este orden:

Espacio	Fecha	Horario	Motivo	Estado
Aula 12	20/05/2027	09:00-10:00	Examen de Matemáticas	Confirmada
Salón de actos	20/05/2027	12:00-14:00	Charla de orientación	Pendiente
Aula 12	24/05/2027	09:00-10:00	Recuperación	Pendiente
Pista polideportiva	24/05/2027	11:15-12:10	Torneo de recreo	Cancelada

- El formulario de nueva reserva tiene los campos «Espacio», «Fecha», «Hora de inicio», «Hora de fin» y «Motivo»; la lista de espacios no incluye «Aula 3». Con el Aula 12 y la fecha 20/05/2027: de 09:30 a 10:30 muestra «El espacio ya tiene una reserva confirmada que se solapa con ese horario.»; de 11:00 a 10:30, «La hora de fin debe ser posterior a la de inicio.»; de 11:00 a 12:00 guarda la reserva y lleva a «Mis reservas», que ahora tiene 5 filas, la nueva como «Pendiente».
- «Salir» vuelve a `/` y el menú muestra «Entrar». Escribir `/cuentas/logout/` en la barra de direcciones devuelve `405 Method Not Allowed`.
- `peticiones.log` muestra ahora `profe1` en las peticiones hechas con la sesión iniciada.

Errores frecuentes

Síntoma	Causa
`/reservas/nueva/` redirige a `/accounts/login/` (404)	Falta `LOGIN_URL = "login"`
`TemplateDoesNotExist: registration/login.html`	La plantilla está en `templates\reservas\registration\` en lugar de `templates\registration\`
`IntegrityError: NOT NULL constraint failed: reservas_reserva.usuario_id`	No se asigna el usuario en `form_valid`
Error 403 «La verificación CSRF ha fallado» al enviar	Falta `{% csrf_token %}` en el formulario
«Salir» da 405	Es un enlace (`<a href>`) en lugar de un formulario POST
Tras un error, la fecha del formulario aparece vacía	Falta `format="%Y-%m-%d"` en el `DateInput`
«Mis reservas» muestra las de todos	`get_queryset` no filtra por `self.request.user`

Fase 9. API REST

Qué vais a conseguir. La API de la aplicación con Django REST Framework: los recursos `/api/espacios/` y `/api/reservas/` con todas sus operaciones, filtros, la consulta de ocupación de la fase 7 y las mismas reglas de negocio que el formulario. De momento os identificaréis con usuario y contraseña (autenticación básica); en la fase 10 se cambiará por tokens.

Antes de empezar. Capítulo 7 (REST, serializadores, `validate()` reutilizando `clean()`, ViewSets, router, filtros, códigos de estado).

Pasos

1. Configurad DRF al final de `settings.py`. No se configura paginación: los listados devuelven **listas JSON**, que es lo que esperará la aplicación Vue de la Unidad 3.

```
# config/settings.py (añadido al final)
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.BasicAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
}
```

2. Escribid `reservas/serializers.py`. **Especificación:**

   - `EspacioSerializer`: campos `id`, `nombre`, `tipo`, `aforo` y `activo`.
   - `ReservaSerializer`: campos `id`, `espacio` (el `id` del espacio), `espacio_nombre` (nombre del espacio, solo lectura), `usuario` (nombre de usuario, solo lectura), `fecha`, `hora_inicio`, `hora_fin`, `motivo`, `estado` y `creada`; `estado` y `creada` son de solo lectura.
   - Las reglas de negocio de la fase 5 se aplican también en la API: el método `validate()` construye una reserva con los datos recibidos (sobre una copia de la reserva existente si es una modificación) y llama a su `clean()`, como `IncidenciaSerializer` en el apartado 7.3. Ojo: el error de solape es **general**, no de un campo, así que no tiene `message_dict`; convertidlo con `error.message_dict if hasattr(error, "error_dict") else error.messages`. Así el error de solape llega al cliente en `non_field_errors`.

3. Escribid `reservas/api.py`. **Especificación:**

   - `EspacioViewSet` (`ModelViewSet`):
     - Filtros `?tipo=aula` y `?activo=true` o `?activo=false`, en `get_queryset()`.
     - Borrar un espacio con reservas responde **409 Conflict** con `{"detail": "El espacio tiene reservas y no se puede borrar."}` en lugar de un error 500 (como `CategoriaViewSet`, apartado 7.4).

  - Una acción adicional para la ocupación: `@action(detail=False, url_path="ocupacion")` sobre un método (por ejemplo, `ocupacion_del_dia(self, request)`), que responde en `/api/espacios/ocupacion/?fecha=AAAA-MM-DD`. Devuelve una lista con `id`, `nombre` y `reservas_confirmadas` de cada espacio, usando la función `ocupacion()` de `consultas.py`. Sin el parámetro `fecha`, responde 400 con `{"detail": "Falta el parámetro fecha (AAAA-MM-DD)."}`.
  - `action` se importa de `rest_framework.decorators`.
- `ReservaViewSet` (`ModelViewSet`):
  - Consulta con `select_related` del espacio y del usuario.
  - Filtros `?fecha=AAAA-MM-DD`, `?espacio=<id>` y `?estado=...`, combinables.
  - Al crear, el usuario es el autenticado (`perform_create`).
- **Parámetros mal escritos:** una fecha mal escrita (`?fecha=20-05-2027`) o un espacio que no es un número deben responder 400, no 500. Pista: convertid la fecha con `date.fromisoformat(texto)` dentro de un `try` y, si lanza `ValueError`, lanzad `serializers.ValidationError({"fecha": "Formato de fecha no válido (AAAA-MM-DD)."})`. DRF convierte esa excepción en una respuesta 400 aunque se lance en `get_queryset()` o en una acción. Para el espacio, `texto.isdigit()`.

4. Registrad los ViewSets en un router en `config/urls.py`:

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from reservas.api import EspacioViewSet, ReservaViewSet

router = DefaultRouter()
router.register("espacios", EspacioViewSet, basename="espacio")
router.register("reservas", ReservaViewSet, basename="reserva")

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cuentas/", include("django.contrib.auth.urls")),
    path("api/", include(router.urls)),
    path("api-auth/", include("rest_framework.urls")),
    path("", include("reservas.urls")),
]
```

5. Ejecutad `python manage.py cargar_datos` y probad con `curl.exe` las peticiones de la comprobación.

Comprobación de la fase

Lecturas:

```
> curl.exe http://127.0.0.1:8000/api/
{"espacios":"http://127.0.0.1:8000/api/espacios/","reservas":"http://127.0.0.1:8000/api/reservas/"}
```

```
> curl.exe http://127.0.0.1:8000/api/espacios/
[{"id":1,"nombre":"Aula
 ↪  12","tipo":"aula","aforo":30,"activo":true},{"id":5,"nombre":"Aula 3",...},
 {"id":2,...},{"id":4,...},{"id":3,"nombre":"Salón de
  ↪  actos","tipo":"salon","aforo":200,"activo":true}]

> curl.exe "http://127.0.0.1:8000/api/espacios/ocupacion/?fecha=2027-05-20"
[{"id":1,"nombre":"Aula 12","reservas_confirmadas":2},
 {"id":2,"nombre":"Laboratorio de Física","reservas_confirmadas":0},
 {"id":4,"nombre":"Pista polideportiva","reservas_confirmadas":0},
 {"id":3,"nombre":"Salón de actos","reservas_confirmadas":0}]
```

(Las respuestas reales van en una sola línea.) Para contar los elementos de una lista en PowerShell: `(curl.exe -s "http://127.0.0.1:8000/api/espacios/?activo=true" | ConvertFrom-Json).Count`.

Petición	Resultado
`/api/espacios/?activo=true`	Lista de 4 espacios
`/api/espacios/?tipo=aula`	Lista de 2 espacios
`/api/espacios/ocupacion/?fecha=2027-05-20`	Laboratorio de Física con 1, el primero; los demás con 0
`/api/espacios/ocupacion/`	400 y `{"detail":"Falta el parámetro fecha (AAAA-MM-DD)."}`
`/api/reservas/`	Lista de 6 reservas; cada una con `"espacio":1` (un número), `"usuario":"profe1"` (un nombre) y horas como `"hora_inicio":"09:00:00"`
`/api/reservas/?fecha=2027-05-20`	Lista de 3 reservas
`/api/reservas/?espacio=1`	Lista de 3 reservas
`/api/reservas/?estado=confirmada`	Lista de 3 reservas
`/api/reservas/?fecha=2027-05-20&espacio=1&estado=confirmada`	Lista de 2 reservas
`/api/reservas/?fecha=20-05-2027`	400 y `{"fecha":"Formato de fecha no válido (AAAA-MM-DD)."}`
`/api/reservas/?espacio=abc`	400

Escrituras (`-u` envía usuario y contraseña):

```
> curl.exe -i -d
 ↪  "espacio=1&fecha=2027-05-20&hora_inicio=11:00&hora_fin=12:00&motivo=Prueba"
 ↪  http://127.0.0.1:8000/api/reservas/
> curl.exe -i -u profe2:Clave.2026 -d
 ↪  "espacio=1&fecha=2027-05-20&hora_inicio=09:30&hora_fin=10:30&motivo=Prueba"
 ↪  http://127.0.0.1:8000/api/reservas/
> curl.exe -i -u profe2:Clave.2026 -d
 ↪  "espacio=1&fecha=2027-05-20&hora_inicio=11:00&hora_fin=10:00&motivo=Prueba"
 ↪  http://127.0.0.1:8000/api/reservas/
```

```
> curl.exe -i -u profe2:Clave.2026 -d
 ↪  "espacio=5&fecha=2027-05-20&hora_inicio=12:00&hora_fin=13:00&motivo=Prueba"
 ↪  http://127.0.0.1:8000/api/reservas/
> curl.exe -i -u profe2:Clave.2026 -d
 ↪  "espacio=1&fecha=2027-05-20&hora_inicio=11:00&hora_fin=12:00&motivo=Prueba&estado=confirmada&usu
 ↪  http://127.0.0.1:8000/api/reservas/
```

Petición	Resultado
Sin credenciales	`401 Unauthorized` y `{"detail":"Las credenciales de autenticación no se proveyeron."}`
Solape	`400` y `{"non_field_errors":["El espacio ya tiene una reserva confirmada que se solapa con ese horario."]}`
Horas al revés	`400` y `{"hora_fin":["La hora de fin debe ser posterior a la de inicio."]}`
Espacio inactivo	`400` y `{"espacio":["El espacio no está disponible para reservas."]}`
Correcta	`201 Created` con `"usuario":"profe2"` y `"estado":"pendiente"`, aunque se hayan enviado otros valores
`PATCH` de esa reserva con `hora_inicio=09:45`	`400` con el error de solape
`PATCH` con `motivo=Cambio`	`200 OK`
`DELETE` de esa reserva	`204 No Content`; después, `GET` devuelve `404`
`DELETE /api/espacios/1/` con `-u profe2:Clave.2026`	`409 Conflict` y `{"detail":"El espacio tiene reservas y no se puede borrar."}`

En esta fase, cualquier usuario autenticado puede todavía crear espacios o modificar reservas ajenas: lo corrige la fase 10.

Errores frecuentes

Síntoma	Causa
Error 500 `AttributeError: 'ValidationError' object has no attribute 'error_dict'` al crear una reserva solapada	Se usa `error.message_dict` con un error general
La API acepta reservas solapadas	`validate()` no llama a `clean()` (DRF no lo hace por sí mismo)
`?fecha=20-05-2027` da 500	El texto se pasa directamente a `filter(fecha=...)`
Borrar el espacio 1 da 500 `ProtectedError`	Falta capturar la excepción en `destroy()`

Síntoma	Causa
Los listados devuelven `{"count": ..., "results": [...]}`	Se ha configurado paginación: hay que quitarla
`/api/espacios/ocupacion/` devuelve 404	Falta `detail=False` en `@action` (con `detail=True` la ruta sería `/api/espacios/<id>/ocupacion/`) o falta `url_path="ocupacion"` y el método tiene otro nombre

Fase 10. Seguridad de la API

Qué vais a conseguir. Las medidas de seguridad definitivas: la API se usa con **tokens**, cada recurso tiene sus **permisos** (solo la conserjería gestiona los espacios; las reservas son privadas y solo su propietario las modifica) y se autoriza el origen de la futura aplicación Vue con **CORS**. Además, revisaréis la configuración con `check --deploy`.

Antes de empezar. Capítulo 8 (tokens, permisos de DRF, CSRF, CORS, `check --deploy`).

Pasos

1. Cambiad la autenticación básica por tokens y añadid la ruta para obtenerlos:

   - En `REST_FRAMEWORK`, sustituid `"rest_framework.authentication.BasicAuthentication"` por `"rest_framework.authentication.TokenAuthentication"` (en primer lugar, para que las peticiones sin credenciales reciban 401).
   - En `config/urls.py`, importad `from rest_framework.authtoken.views import obtain_auth_token` y añadid, antes de `path("api/", ...)`, la ruta `path("api/token/", obtain_auth_token, name="api-token"),`.
   - La aplicación `rest_framework.authtoken` ya está instalada y migrada desde la fase 1.

2. Escribid `reservas/permissions.py` con dos permisos. **Especificación:**

   - `EsPersonalOSoloLectura`: cualquiera puede leer (`SAFE_METHODS`); para escribir hay que estar autenticado y tener `is_staff`. Pista: se comprueba en `has_permission`, porque afecta también a las altas (no hay objeto).
   - `EsPropietarioOSoloLectura`: cualquiera puede leer; modificar o borrar solo el usuario de la reserva. Pista: `has_object_permission`, como `EsAutorOSoloLectura` en el apartado 8.5.

3. Aplicad los permisos con `permission_classes`: `EspacioViewSet` con `EsPersonalOSoloLectura`; `ReservaViewSet` con `IsAuthenticated` (las reservas no son públicas, ni siquiera para leer) y `EsPropietarioOSoloLectura`.

4. Configurad CORS: añadid `"corsheaders.middleware.CorsMiddleware"` en `MIDDLEWARE` justo después de `SecurityMiddleware` (en todo caso, antes de `CommonMiddleware`), y al final de `settings.py`:

```
# config/settings.py (añadido al final)
CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]
```

5. Revisad que los formularios de la aplicación tienen `{% csrf_token %}` y que el cierre de sesión es por POST (fase 8). Las contraseñas solo se crean con `create_user()`, `set_password()` o `createsuperuser`.

6. Ejecutad `python manage.py check --deploy` y anotad los avisos para el `README.md` (fase 11).

7. Ejecutad `python manage.py cargar_datos`, guardad los tokens en variables de PowerShell y probad:

```
> $t1 = (curl.exe -s -X POST -d "username=profe1&password=Clave.2026"
 ↪  http://127.0.0.1:8000/api/token/ | ConvertFrom-Json).token
> $t2 = (curl.exe -s -X POST -d "username=profe2&password=Clave.2026"
 ↪  http://127.0.0.1:8000/api/token/ | ConvertFrom-Json).token
> $tc = (curl.exe -s -X POST -d "username=conserje&password=Clave.2026"
 ↪  http://127.0.0.1:8000/api/token/ | ConvertFrom-Json).token
> curl.exe -i http://127.0.0.1:8000/api/reservas/
> curl.exe -i -H "Authorization: Token $t2"
 ↪  "http://127.0.0.1:8000/api/reservas/?fecha=2027-05-20"
> curl.exe -i -H "Authorization: Token $t1" -d "nombre=Aula
 ↪  20&tipo=aula&aforo=20&activo=true" http://127.0.0.1:8000/api/espacios/
> curl.exe -i -H "Authorization: Token $tc" -d "nombre=Aula
 ↪  20&tipo=aula&aforo=20&activo=true" http://127.0.0.1:8000/api/espacios/
> curl.exe -i -H "Origin: http://localhost:5173"
 ↪  http://127.0.0.1:8000/api/espacios/
> curl.exe -i -H "Origin: http://localhost:8080"
 ↪  http://127.0.0.1:8000/api/espacios/
```

Para probar el permiso del propietario, buscad el `id` de una reserva de `profe2` (por ejemplo, la «Práctica de óptica» en `/api/reservas/?fecha=2027-05-24`) e intentad modificarla y borrarla con `$t1`, y después modificarla con `$t2`:

```
> curl.exe -i -X PATCH -H "Authorization: Token $t1" -d "motivo=Cambio"
 ↪  http://127.0.0.1:8000/api/reservas/N/
> curl.exe -i -X DELETE -H "Authorization: Token $t1"
 ↪  http://127.0.0.1:8000/api/reservas/N/
> curl.exe -i -X PATCH -H "Authorization: Token $t2" -d "motivo=Cambio"
 ↪  http://127.0.0.1:8000/api/reservas/N/
```

Comprobación de la fase

Petición	Resultado
`POST /api/token/` correcto	`{"token":"..."}` con 40 caracteres; repetido, el mismo token
`POST /api/token/` con contraseña incorrecta	`400` y `{"non_field_errors":["No puede iniciar sesión con las credenciales proporcionadas."]}`
`GET /api/reservas/` sin token	`401 Unauthorized`, `WWW-Authenticate: Token` y `{"detail":"Las credenciales de autenticación no se proveyeron."}`
`GET /api/reservas/` con `-u profe1:Clave.2026`	`401` (la autenticación básica ya no está activa)
`GET /api/reservas/?fecha=2027-05-20` con `$t2`	`200` y lista de 3 reservas
`GET /api/espacios/` sin token	`200`
`POST /api/espacios/` con `$t1`	`403 Forbidden` y `{"detail":"Usted no tiene permiso para realizar esta acción."}`
`POST /api/espacios/` con `$tc`	`201 Created` (borrad después «Aula 20» con `DELETE` y `$tc`: `204`)
`PATCH` y `DELETE` de una reserva de `profe2` con `$t1`	`403 Forbidden`
`PATCH` de esa reserva con `$t2`	`200 OK`
`Origin: http://localhost:5173`	La respuesta incluye `access-control-allow-origin: http://localhost:5173`
`Origin: http://localhost:8080`	`200 OK` sin ninguna cabecera `access-control-`
`check --deploy`	`System check identified 7 issues (0 silenced).`: W004, W008, W009, W012, W016, W018 y W020

Si al crear un espacio enviáis los datos como formulario y omitís `activo`, se crea **inactivo**: en los datos de formulario, un campo booleano que no se envía cuenta como falso. Por eso el ejemplo envía `activo=true`.

Errores frecuentes

Síntoma	Causa
Las peticiones sin token reciben 403 en lugar de 401	`SessionAuthentication` está la primera en `DEFAULT_AUTHENTICATION_CLASSES`
`GET /api/reservas/` sin token devuelve 200	Falta `IsAuthenticated` en `ReservaViewSet`

Síntoma	Causa
`profe1` puede crear espacios	El permiso de personal se comprueba en `has_object_permission` (que no se ejecuta en las altas)
`profe1` modifica reservas ajenas	Falta `EsPropietarioOSoloLectura` o `permission_classes` sustituye a los permisos sin incluirlo
No aparece `access-control-allow-origin`	`CorsMiddleware` falta o está después de `CommonMiddleware`, o el origen no coincide exactamente
`/api/token/` devuelve 404	La ruta está después de `path("", include(...))` o mal escrita

Fase 11. Repaso final y entrega

Qué vais a conseguir. Comprobar que la práctica cumple todos los requisitos, documentarla y entregarla.

Pasos

1. Reconstruid el proyecto como lo hará el profesor, en una copia limpia: copiad la carpeta `reservas` **sin** `.venv` ni `db.sqlite3` a otra carpeta y ejecutad en ella:

   ```
   > python -m venv .venv
   > .\.venv\Scripts\Activate.ps1
   > python -m pip install -r requirements.txt
   > python manage.py migrate
   > python manage.py check
   > python manage.py makemigrations --check
   > python manage.py cargar_datos
   > python manage.py runserver
   ```

   `check` debe responder sin avisos y `makemigrations --check`, `No changes detected`.

2. Repasad uno a uno los **requisitos mínimos para el Apto** (apartado siguiente) con los datos de `cargar_datos` recién ejecutada. Cada requisito indica la fase en la que se consigue: si alguno falla, volved a la comprobación de esa fase.

3. Si habéis cambiado de paquetes, volved a generar `requirements.txt` con `pip freeze`.

4. Escribid el `README.md` de la carpeta del proyecto con el contenido que piden los **entregables**.

5. Entregad según el apartado de entregables.

Requisitos mínimos para el Apto

Cada requisito se comprueba con los datos de `cargar_datos` recién ejecutada sobre una base de datos nueva (espacios con los identificadores 1 a 5). **Basta que falle uno para que la práctica sea No Apta.**

N.º	Comprobación	Fase	CE
1	En un equipo limpio, `pip install -r requirements.txt`, `python manage.py migrate` y `python manage.py check` terminan sin errores, y `python manage.py makemigrations --check` responde `No changes detected`	1, 5, 11	`RA1.a`, `RA1.d`
2	La entrega no contiene `.venv`, `db.sqlite3` ni carpetas `__pycache__`; la estructura es `reservas/config/` y `reservas/reservas/`; el panel de administración aparece en español	1, 11	`RA1.a`
3	Los modelos tienen los campos, tipos, opciones y valores por defecto de la fase 5, y `showmigrations reservas` muestra todas sus migraciones aplicadas	5	`RA1.d`
4	`python manage.py cargar_datos` muestra `Cargados 5 espacios y 6 reservas` y, ejecutada dos veces, no duplica usuarios ni espacios	6	`RA1.a`, `RA1.e`
5	`/` lista los 4 espacios activos (no «Aula 3») con tipo y aforo; `/espacios/1/` muestra las dos reservas confirmadas del 20/05/2027; `/espacios/999/` devuelve 404; todas las páginas comparten la plantilla base	3, 7	`RA1.b`
6	Sin sesión, `/reservas/nueva/` y `/reservas/mias/` redirigen a `/cuentas/login/?next=...`. Como `profe1`, «Mis reservas» muestra exactamente sus 4 reservas, ordenadas por fecha y hora	8	`RA1.b`, `RA1.g`
7	Como `profe1`, el formulario solo ofrece espacios activos; reservar el Aula 12 el 20/05/2027 de 09:30 a 10:30 muestra el error de solape; de 11:00 a 10:30, el error de la hora de fin; de 11:00 a 12:00 se guarda como pendiente de `profe1` y redirige a «Mis reservas»	5, 8	`RA1.b`, `RA1.d`

N.º	Comprobación	Fase	CE
8	Cada petición añade una línea a `peticiones.log` con método, ruta, código y usuario (`anonimo` sin sesión)	4	`RA1.c`
9	En el panel, el listado de reservas tiene los filtros y el buscador pedidos; si se intenta guardar desde el panel una reserva **confirmada** del Aula 12 el 20/05/2027 de 09:30 a 10:30, el panel muestra el error de solape y no la guarda	6	`RA1.d`, `RA1.e`
10	`GET /api/espacios/ocupacion/?fecha=2027-05-20` devuelve Aula 12 con 2 y los otros tres espacios activos con 0, en el orden pedido; con `fecha=2027-05-24`, Laboratorio de Física con 1 el primero	7, 9	`RA1.e`
11	Con un token válido, `/api/reservas/?fecha=2027-05-20` devuelve una lista de 3 reservas; `?espacio=1`, 3; `?estado=confirmada`, 3; los tres filtros combinados, 2; `?fecha=20-05-2027` devuelve 400	9	`RA1.e`, `RA1.f`
12	La API tiene la raíz `/api/` con los dos recursos, listados sin paginar (la respuesta empieza por `[`) y responde con los códigos correctos: 201 al crear, 204 al borrar, 400 con datos inválidos, 404 si no existe y 409 al borrar el espacio 1	9	`RA1.f`
13	`POST /api/reservas/` con el token de `profe2` aplica las tres reglas de negocio (400 con los mensajes de la fase 5; el de solape en `non_field_errors`) y, con datos correctos, crea la reserva con `"usuario": "profe2"` y `"estado": "pendiente"` aunque se envíen otros valores en esos campos	9	`RA1.f`, `RA1.g`
14	`POST /api/token/` con credenciales correctas devuelve un token de 40 caracteres y con incorrectas, 400. `GET /api/reservas/` sin token devuelve 401. `GET /api/espacios/` sin token devuelve 200	10	`RA1.g`

N.º	Comprobación	Fase	CE
15	`POST /api/espacios/` con el token de `profe1` devuelve 403 y con el de `conserje`, 201. `PATCH` o `DELETE` de una reserva de `profe2` con el token de `profe1` devuelven 403	10	`RA1.g`
16	Una petición con la cabecera `Origin: http://localhost:5173` recibe `Access-Control-Allow-Origin: http://localhost:5173`; con `Origin: http://localhost:8080`, no recibe ninguna cabecera `Access-Control-`	10	`RA1.g`
17	El `README.md` incluye las instrucciones de arranque, las capturas pedidas y la explicación de los avisos de `check --deploy`	11	`RA1.g`

Calidad técnica (no decide el Apto)

Estos aspectos no deciden el Apto, pero se valorarán y **se preguntarán en el examen del RA1** sobre vuestra propia práctica:

- La regla de solape está escrita **una sola vez** (en el modelo) y la reutilizan el formulario, el panel y el serializador.
- Ausencia de N+1 consultas: `select_related` en los listados de reservas (vistas, panel y API) y una sola consulta en la ocupación.
- Posición de `RegistroPeticionesMiddleware` y de `CorsMiddleware` en `MIDDLEWARE`, y por qué.
- Campos enumerados explícitamente en formularios y serializadores (sin `"__all__"`), y campos que nunca debe enviar el cliente.
- Uso de nombres de ruta (`{% url %}`, `reverse`, `get_absolute_url`) en lugar de direcciones escritas a mano.
- Códigos de estado HTTP adecuados en cada caso, sin errores 500 ante datos incorrectos.
- Restricción en la base de datos que garantice que `hora_fin` es posterior a `hora_inicio` (`CheckConstraint`), además de la validación del modelo.
- Código según PEP 8, nombres claros, sin código muerto ni `print()` de depuración.
- Historial de Git con un *commit* por fase como mínimo y mensajes descriptivos (si entregáis repositorio).

Entregables

En el plazo indicado en el aula virtual de EducamosCLM, una de estas dos opciones:

- Enlace a un **repositorio Git** (GitHub o GitLab, privado con acceso para el profesor), o

- Un fichero **`UT2-P01-<vuestro-identificador>.zip`** con la carpeta `reservas`.

En los dos casos, **sin** `.venv`, `db.sqlite3`, `peticiones.log` ni `__pycache__`. La carpeta debe contener:

- El proyecto completo, `requirements.txt` y `.gitignore`.
- Un **`README.md`** con:
  - Los pasos exactos para arrancar el proyecto desde cero en Windows (entorno virtual, dependencias, migraciones, `cargar_datos`, `runserver`).
  - Capturas de: el listado de espacios; «Mis reservas» de `profe1`; el error de solape en el formulario; el panel de administración con el filtro de reservas; la respuesta de `/api/espacios/ocupacion/?fecha=2027-05-20`; un 403 y un 401 de la API con `curl.exe -i`; y las últimas líneas de `peticiones.log`.
  - Las órdenes `curl.exe` que habéis usado para probar la API.
  - La explicación de cada aviso de `python manage.py check --deploy`: qué significa y cómo lo resolveríais en un servidor de producción (no hace falta resolverlos ahora).

Si la práctica es **No Apta**, recibiréis la lista de requisitos que han fallado y podréis hacer una segunda entrega en el plazo indicado en el aula virtual.

Uso de inteligencia artificial

Esta práctica es un trabajo individual evaluable. Según la programación didáctica del módulo, en caso de indicio o sospecha fundada de uso no autorizado de inteligencia artificial (IA) u otros medios fraudulentos en su elaboración, el profesorado podrá convocaros a una **prueba de verificación individual** (oral o escrita) sobre los contenidos entregados. La no superación de dicha prueba o la negativa injustificada a realizarla conllevará la calificación de **0** en la actividad, sin perjuicio de las medidas disciplinarias recogidas en las normas de convivencia del centro.

Además, en el examen del RA1 se os preguntará por las decisiones técnicas de vuestra propia práctica: tenéis que poder explicar cada línea que entreguéis.